Sarah Benavides

CS 362 Lab 15

4/25/2026

Malware Analysis

Part 1: Non-technical summary

This lab introduced the basic ideas for analyzing malicious software in a safe, controlled manner. We began by reviewing what malware is and the different types, such as viruses, worms, Trojan horses, ransomware, and more. We also reviewed how malware spreads and affects systems. We then focused on static analysis, which examines a suspicious file without running it. Next, the structure of Windows executable files was explored to understand how programs are organized and what information they contain. By using simple tools, we inspected important details like file sections, embedded data, and the libraries a program relies on, which can reveal what the malware is capable of doing.

This lab also demonstrates how to extract readable text from a program to uncover hidden clues such as network activity or system behavior. In addition, this lab also introduced basic techniques for identifying how malware may disguise itself to look like normal files. Lastly, the lab briefly explored dynamic analysis by using online tools to observe how a suspicious file behaves in a secure environment. Overall, this lab builds a great foundation for understanding and safely investigating malware.

Part 2: Technical summary

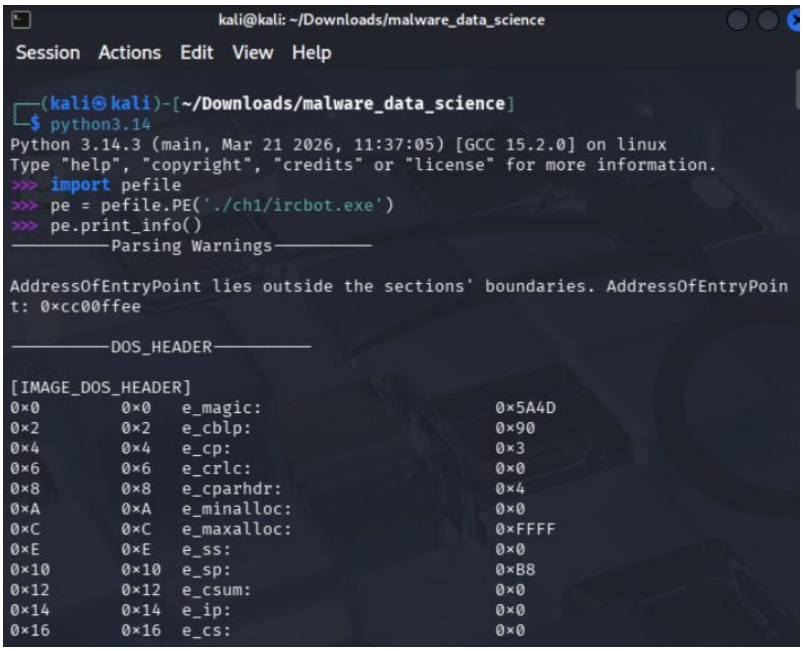
In this lab, we concentrated on analyzing different types of malware. For the first part, we downloaded a zip file from the website https://www.malwaredatascience.com/code-and-data and unzipped it in the downloads folder. Before we dive into the analysis, we need to understand what malware actually is. Malware is short for malicious software and is basically any software that is created and used by threat actors to gain access or cause harm to a system. The two main types of malware analysis are static and dynamic analysis. Static analysis is performed by analyzing a program file's code, images, strings, and other stored information. On the other hand, dynamic analysis involves running malware in a safe and isolated environment to analyze its behavior.

The most common types of malware are as follows: viruses, Trojan horses, computer worms, rootkits, botnets, adware, spyware, and ransomware. A virus is a computer code that can replicate itself by modifying other files or programs. The inserted code is capable of further replication. Replication relies on the user, such as clicking on an email attachment, sharing a USB drive, etc. The Trojan horse is a bit different. This malware is a program that appears to be benign, but in fact, it is actually doing some malicious actions. Slightly similar to viruses, computer worms spread by self-replication, yet they are capable of spreading copies of themselves without needing to infect other programs. In one of the last labs, we touched on rootkits, and we know that they are programs designed to gain administrator-level or root-level access over an operating system without being detected. The next malware to look at is botnets, which are large-scale attacks that can be performed by a network of compromised computers and

controlled by the bot herder. The type of malware people typically see daily is adware. Adware is a software that displays advertisements on a user's screen against their consent. Another type of malware that acts without the user's consent is spyware. This software is installed on a user's computer without their consent and gathers information about the user, their communication, or their computer usage. The last malware we visited is ransomware. Typically, companies and organizations deal with this malware the most. This malware basically encrypts a device's data. The keys used to decrypt the data are exchanged for money or something else of great value.

In the third part of this lab, we focused on the Portable Executable File Format (PE file format). The PE file format is used by Windows operating systems for several file types, including executable files (.exe), dynamic link libraries (DLL), ActiveX control (.ocx), system files (.sys), and kernel drivers (.drv). The PE format includes information to instruct the operating system how to load the program into memory, in addition to several sections that contain the executable's actual data. To begin this static analysis, we first look at the DOS header. This header is present for compatibility reasons. Two important values this header contains are e_magic and e_lfanew. E_magic is set to the value 0x5A4D, and e_lfanew contains the offset of the PE header.

In order to analyze the information of PE files, I inserted the command **python3.14** into the terminal. This shifted the terminal into an interactive shell. I was then able to input the code below.



Once I hit enter after the last line of code, the lengthy output was displayed. The output included any parsing warnings, the DOS headers, NT headers, file header, optional header, section headers, directories, and imported symbols.

The most important value within the NT headers is the signature. To list this, we entered the code below.

```
>>> print("[*] Signature: " + hex(pe.NT_HEADERS.Signature))
[*] Signature: 0×4550
```

In this output, the signature is set to 0x4550. The next header we analyze is the file header. This structure contains information about the number of sections and the time at which the file author compiled it. To list this information, we input the code below.

```
>>> for field in pe.FILE_HEADER.dump():
...     print(field)
...
[IMAGE_FILE_HEADER]
0×E4       0×0   Machine:                        0×14C
0×E6       0×2   NumberOfSections:               0×5
0×E8       0×4   TimeDateStamp:                  0×4F79D506 [Mon Apr  2 16:34:
14 2012 UTC]
0×EC       0×8   PointerToSymbolTable:           0×0
0×F0       0×C   NumberOfSymbols:                0×0
0×F4       0×10  SizeOfOptionalHeader:           0×E0
0×F6       0×12  Characteristics:                0×10F
```

This output lists four different columns, including the hexadecimal offset, the field size in hexadecimal, the field name, and the value.

Another header we touch on is the optional header. Even though the name has the word "optional" in it, it contains very important information, such as the program's entry point in the PE file. I entered the code below to list the optional header's output.

```
>>> for field in pe.OPTIONAL_HEADER.dump():
...     print(field)
...
[IMAGE_OPTIONAL_HEADER]
0×F8       0×0   Magic:                          0×10B
0×FA       0×2   MajorLinkerVersion:            0×6
0×FB       0×3   MinorLinkerVersion:            0×0
0×FC       0×4   SizeOfCode:                    0×32A00
0×100      0×8   SizeOfInitializedData:         0×64200
0×104      0×C   SizeOfUninitializedData:       0×0
0×108      0×10  AddressOfEntryPoint:           0×CC00FFEE
0×10C      0×14  BaseOfCode:                    0×1000
0×110      0×18  BaseOfData:                    0×1000
0×114      0×1C  ImageBase:                     0×400000
0×118      0×20  SectionAlignment:              0×1000
0×11C      0×24  FileAlignment:                 0×200
0×120      0×28  MajorOperatingSystemVersion:   0×4
0×122      0×2A  MinorOperatingSystemVersion:   0×0
0×124      0×2C  MajorImageVersion:             0×0
0×126      0×2E  MinorImageVersion:             0×0
0×128      0×30  MajorSubsystemVersion:         0×4
0×12A      0×32  MinorSubsystemVersion:         0×0
0×12C      0×34  Reserved1:                     0×0
0×130      0×38  SizeOfImage:                   0×9A000
0×134      0×3C  SizeOfHeaders:                 0×1000
0×138      0×40  CheckSum:                      0×0
```

This output was similar to the file header output because both included the hexadecimal offset, field size, field name, and the values. To list the specific names, sizes, and addresses of the data table, we entered the code below.

```
>>> for val in pe.OPTIONAL_HEADER.DATA_DIRECTORY:
...     print("Name: " + val.name + " | Size : " + str(val.Size) + " | Addre\
ss : " + hex(val.VirtualAddress))
...
Name: IMAGE_DIRECTORY_ENTRY_EXPORT | Size : 0 | Address : 0x0
Name: IMAGE_DIRECTORY_ENTRY_IMPORT | Size : 60 | Address : 0x96000
Name: IMAGE_DIRECTORY_ENTRY_RESOURCE | Size : 0 | Address : 0x0
Name: IMAGE_DIRECTORY_ENTRY_EXCEPTION | Size : 0 | Address : 0x0
Name: IMAGE_DIRECTORY_ENTRY_SECURITY | Size : 0 | Address : 0x0
Name: IMAGE_DIRECTORY_ENTRY_BASERELOC | Size : 0 | Address : 0x0
Name: IMAGE_DIRECTORY_ENTRY_DEBUG | Size : 0 | Address : 0x0
Name: IMAGE_DIRECTORY_ENTRY_COPYRIGHT | Size : 0 | Address : 0x0
Name: IMAGE_DIRECTORY_ENTRY_GLOBALPTR | Size : 0 | Address : 0x0
Name: IMAGE_DIRECTORY_ENTRY_TLS | Size : 0 | Address : 0x0
Name: IMAGE_DIRECTORY_ENTRY_LOAD_CONFIG | Size : 0 | Address : 0x0
Name: IMAGE_DIRECTORY_ENTRY_BOUND_IMPORT | Size : 0 | Address : 0x0
Name: IMAGE_DIRECTORY_ENTRY_IAT | Size : 0 | Address : 0x0
Name: IMAGE_DIRECTORY_ENTRY_DELAY_IMPORT | Size : 0 | Address : 0x0
Name: IMAGE_DIRECTORY_ENTRY_COM_DESCRIPTOR | Size : 0 | Address : 0x0
Name: IMAGE_DIRECTORY_ENTRY_RESERVED | Size : 0 | Address : 0x0
```

The last type of header to concentrate on is the section header. This contains information about the sections of the data. To display all the different types of sections, we input the code below.

```
>>> for sections in pe.sections:
...     print(" Virtual address: " + hex(section.VirtualAddress))
...     print(" Virtual size " + hex(section.Misc_VirtualSize))
...     print(" Size of raw data " + hex(section.SizeOfRawData) + '\n')
...
 Virtual address: 0x1000
 Virtual size 0x32830
 Size of raw data 0x32a00

 Virtual address: 0x1000
 Virtual size 0x32830
 Size of raw data 0x32a00

 Virtual address: 0x1000
 Virtual size 0x32830
 Size of raw data 0x32a00

 Virtual address: 0x1000
 Virtual size 0x32830
 Size of raw data 0x32a00

 Virtual address: 0x1000
 Virtual size 0x32830
 Size of raw data 0x32a00
```

This output includes the virtual address, virtual size, as well as the size of the raw data. We also have the ability to display the content of any of the sections by entering the index.

```
>>> print(pe.sections[2])
[IMAGE_SECTION_HEADER]
0x228      0x0   Name:                          .data
0x230      0x8   Misc:                          0x5CFF8
0x230      0x8   Misc_PhysicalAddress:          0x5CFF8
0x230      0x8   Misc_VirtualSize:              0x5CFF8
0x234      0xC   VirtualAddress:                0x39000
0x238      0x10  SizeOfRawData:                 0x2A00
0x23C      0x14  PointerToRawData:              0x37200
0x240      0x18  PointerToRelocations:          0x0
0x244      0x1C  PointerToLinenumbers:          0x0
0x248      0x20  NumberOfRelocations:           0x0
0x24A      0x22  NumberOfLinenumbers:           0x0
0x24C      0x24  Characteristics:               0xC0000040
```

This output displays the image section header for a specific section of the malware file.

The next file type we want to analyze and focus on is the dynamic link library (DLL). To list the DLLs that a binary will load and the function calls performed in each of those DLLs, we input the following code into the terminal.

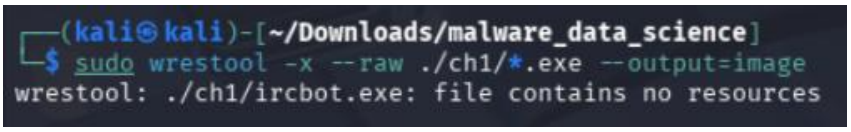
```
>>> for entry in pe.DIRECTORY_ENTRY_IMPORT:
...     print(entry.dll.decode('utf-8'))
...     for fnc in entry.imports:
...         print("| ", fnc.name.decode('utf-8'))
...
KERNEL32.DLL
|  GetLocalTime
|  ExitThread
|  CloseHandle
|  WriteFile
|  CreateFileA
|  ExitProcess
|  CreateProcessA
|  GetTickCount
|  GetModuleFileNameA
|  GetSystemDirectoryA
|  Sleep
|  GetTimeFormatA
|  GetDateFormatA
|  GetLastError
|  CreateThread
|  GetFileSize
|  GetFileAttributesA
|  FindClose
|  FileTimeToSystemTime
|  FileTimeToLocalFileTime
|  FindNextFileA
|  FindFirstFileA
|  SetConsoleCtrlHandler
|  GetStringTypeA
|  GetStringTypeW
|  SetEndOfFile
USER32.dll
|  MessageBoxA
```

This output lists two different DLLs along with a long list of function calls. If we needed to list the DLLs that another binary will load, we can use the code below.

```
>>> import pefile
>>> pe = pefile.PE('./ch1/fakepdfmalware.exe')
>>> for entry in pe.DIRECTORY_ENTRY_IMPORT:
...     print(entry.dll.decode('utf-8'))
...     for fnc in entry.imports:
...         print("| ", fnc.name.decode('utf-8'))
...
KERNEL32.dll
|  CreateDirectoryA
|  ExpandEnvironmentStringsA
|  WaitForSingleObject
|  GetStartupInfoA
|  SetCurrentDirectoryA
|  WriteFile
|  FreeResource
|  GetTickCount
|  SizeofResource
|  LoadResource
|  FindResourceA
|  GetModuleHandleA
|  MoveFileExA
|  lstrcpyA
|  IsDebuggerPresent
|  LoadLibraryA
|  GetProcAddress
|  CreateProcessA
ADVAPI32.dll
|  RegQueryValueExA
|  RegCloseKey
|  CryptEncrypt
|  CryptAcquireContextA
|  CryptCreateHash
|  CryptHashData
|  CryptDeriveKey
|  CryptDestroyHash
|  CryptDecrypt
|  RegOpenKeyA
SHELL32.dll
|  ShellExecuteA
LZ32.dll
|  LZOpenFileA
|  LZClose
|  LZCopy
MSVCRT.dll
|  strcmp
|  free
|  fclose
|  fwrite
|  fread
|  malloc
|  fopen
|  memcpy
|  strlen
|  _beginthreadex
```

This output lists five different DLLs with all their function calls underneath them. This list was lengthy; therefore, I did not include the entry thing.

Malware can be extremely intricate and can disguise itself as Word or PDF documents that people think are generally safe. In this last step of part three, we will retrieve the executable images and use two commands from the **icoutils** library that contain the commands **wrestool** and **icotool**. To begin, we install the **icoutils** library by using the following command: **sudo apt-get install icoutils**. After successfully installing this library, we use the command **mkdir image** to create a folder to store images. The next thing we want to accomplish is to extract the executable icon of several files using the command **sudo wrestool -x --raw ./ch1/*.exe –output=images**. Before the extraction occurs, I change the directory to **/home/kali/Download/malware_data_science**.

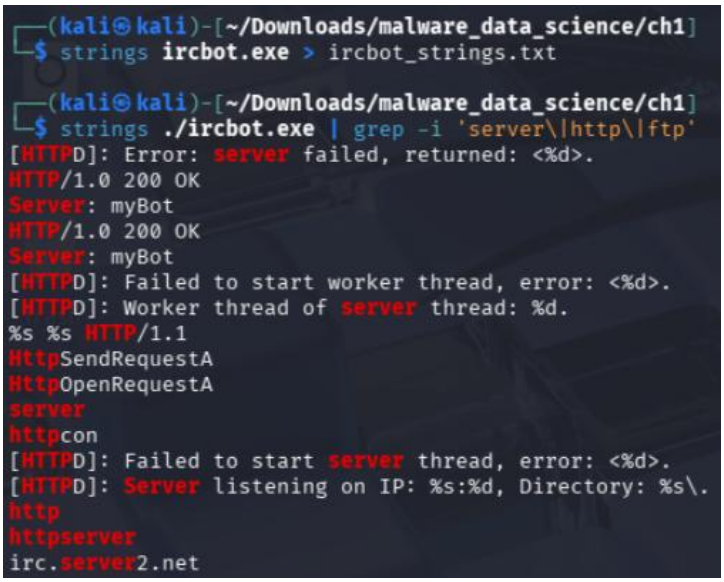


This output indicates that the file ircbot.exe contains no resources. When we enter the command **ls**, an image folder is listed.

To convert the .ico file to a .png image file, we use the command **icotool -x -o images images/*.ico.**

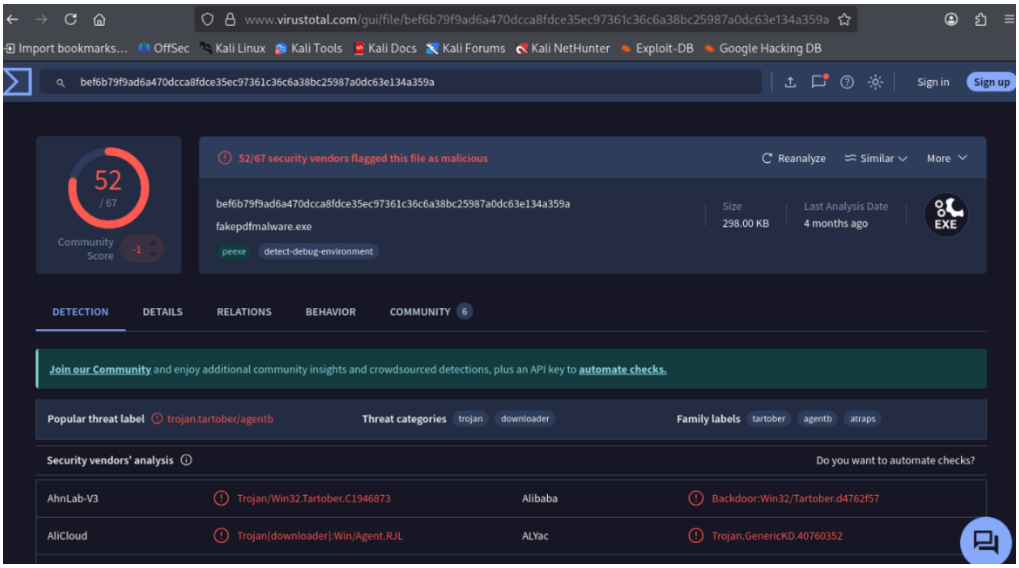
I unfortunately was unable to convert the image file to a .png file because the file could not be opened.

In the fourth part of this lab, we examined malware strings. Strings are printable characters within a program binary. We must remember that it is valuable to analyze the strings of suspicious software to extract information such as HTTP connections, FTP connections, IP addresses, servers' and hosts' names, embedded scripts, HTML requests, the programming language used to create the malware, and so on. To begin this analysis, we input the command **string ircbot.exe > ircbot_strings.txt** to redirect the information from the .exe file and store it in the new text file we created. Next, we use the following command to save everything that is being filtered in real-time to the text file we created, which displays the most suspicious strings.

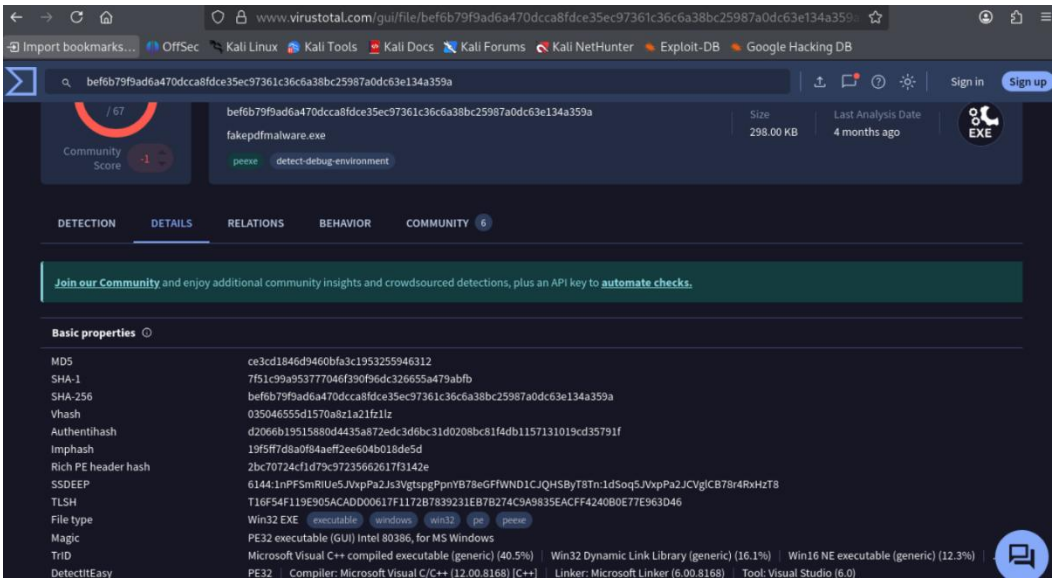


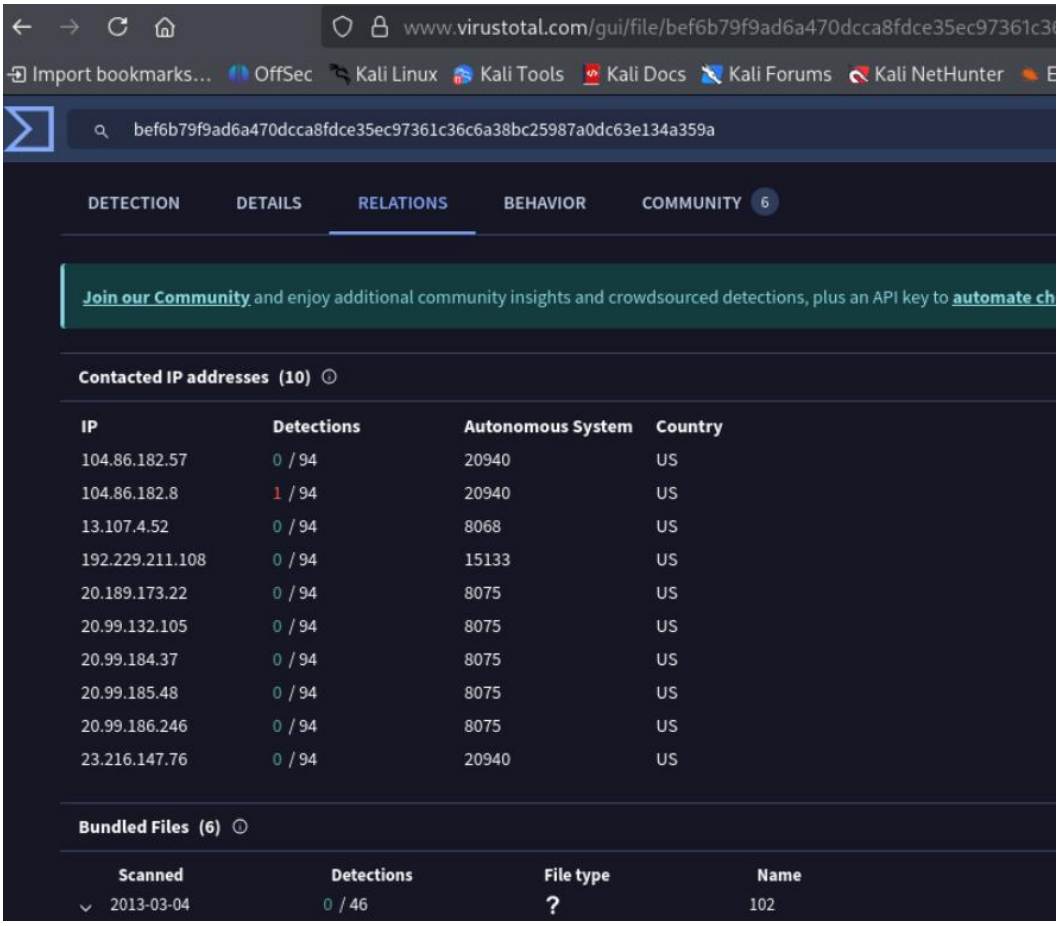
This output listed only the most questionable strings.

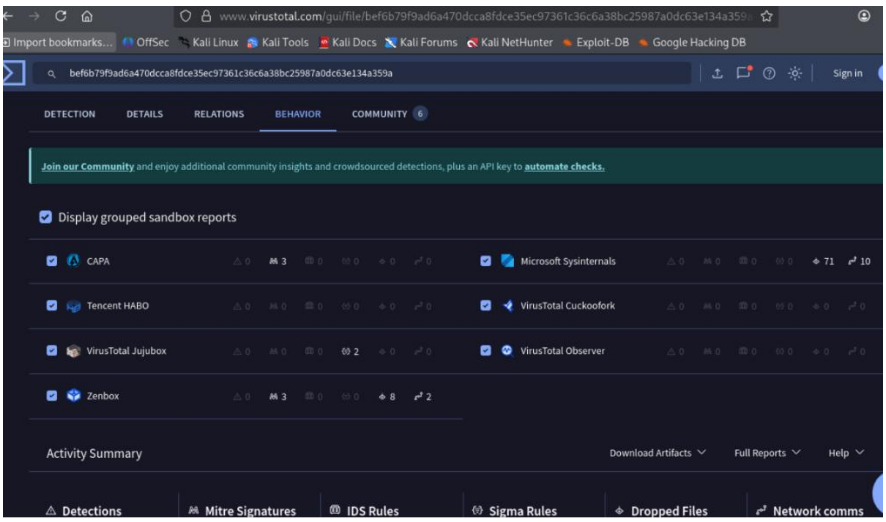
The last part of this lab touched on dynamic malware analysis. We entered the website www.virustotal.com and input the file fakepdfmalware.exe. The website performed an analysis of this suspicious program and created a report.



In this report, a community score of 52 out of 67 was given. This means that 52 out of 67 vendors flagged this file as malware. The main report page also includes details such as the size of the file, the last time the file was analyzed, and which vendors analyzed the file. When clicking over to the details tab, basic properties are listed, including many hashing algorithms. The names, history, and portable executable information from the program are also displayed. The next tab over, relations, lists the relationships and connections the program has made. Lastly, the behavior tab displays grouped sandbox reports and a lengthy activity summary of the program.







The last website we visited was https://hybrid-analysis.com/. After entering the same program as above, an analysis report populates. This report states the name, size, and type of file, as well as listing the program as malicious and rates the threat score as 100/100. The anti-virus results and the Falcon Sandbox report both also deem this program malicious. The last few sections list the relations in which there is one execution parent, as well as the incident response.

Part 3: References

1. Lab 15: Malware Analysis